equals() and hashCode(): The Contract

9 min read Updated 28 Jul 2026 Topic 10 of 65

Bookmark Mark done

Why it matters

This is the single highest-yield topic in a Java interview. It is asked at every level, it has a precise correct answer, and getting it wrong causes one of the most baffling bugs in the language: an object you definitely put into a `HashMap` cannot be found again. Every collection in Chapter 4 depends on this contract, and so does every JPA entity in Chapter 12.

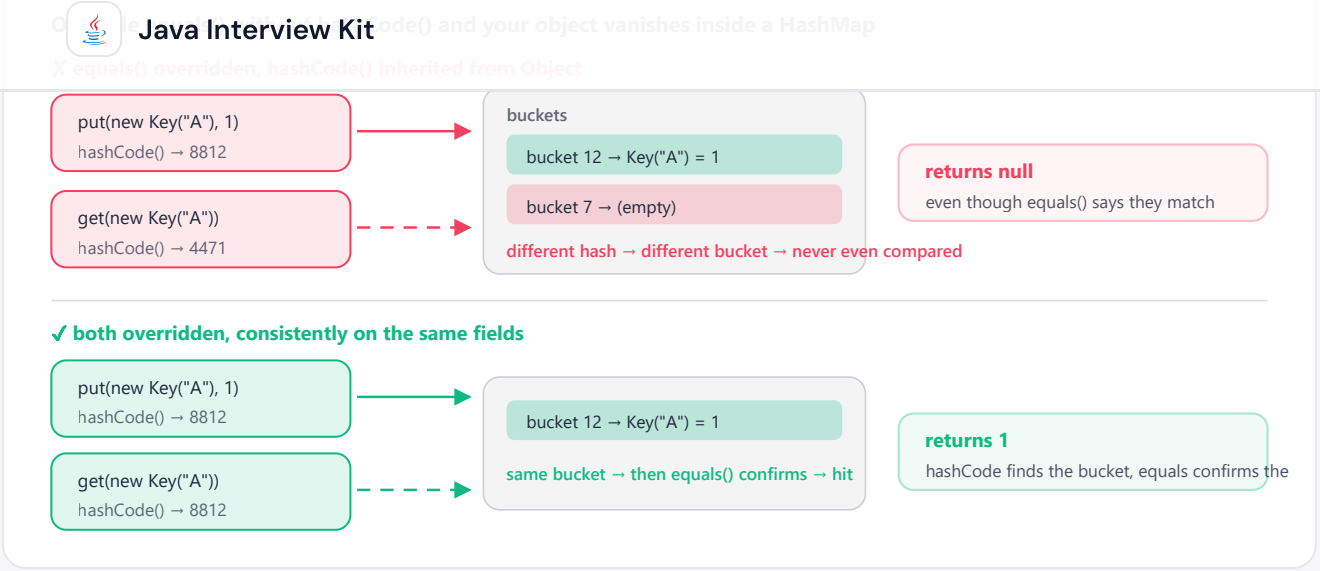
The concept, simply

`Object` gives every class two methods with default implementations that are almost always wrong for your class:

- `equals(Object)` — defaults to `==`, i.e. "is this the same object?"
- `hashCode()` — defaults to a value derived from the object's identity, so two equal-in-meaning objects get different numbers.

The contract binding them is short and absolute: **if two objects are `equals`, they must have the same `hashCode`**. The converse is not required — two unequal objects may share a hash code, which is an ordinary collision.

Override one without the other and you break it. Overriding `equals` alone is the disaster case:



A hash lookup is two steps: hashCode picks the bucket, equals confirms the key. If hashCode disagrees, the map never reaches step two — your entry is in the table and unreachable.

How it works

The full `equals` contract — five properties, all testable:

Property	Meaning
Reflexive	`x.equals(x)` is true
Symmetric	`x.equals(y)` ⟺ `y.equals(x)`
Transitive	x=y and y=z ⟹ x=z
Consistent	Repeated calls give the same answer while the objects are unchanged
Non-null	`x.equals(null)` is false, never an exception

And the `hashCode` contract:

- Equal objects **must** have equal hash codes.
- Unequal objects *should* ideally differ, for performance — but need not.
- The value must be stable while the object's `equals`-relevant fields are unchanged.

The lookup path makes the reason concrete. `map.get(key)` computes `key.hashCode()`, derives a bucket index from it, and *only then* walks that one bucket

Code examples

The bug, reproduced in six lines:

JAVA Copy

```
class Key {
    final String id;
    Key(String id) { this.id = id; }

    @Override public boolean equals(Object o) {
        return o instanceof Key k && id.equals(k.id);
    }
    // hashCode() NOT overridden — inherited identity hash
}

Map<Key, String> map = new HashMap<>();
map.put(new Key("A"), "value");
System.out.println(map.get(new Key("A")));            // null   ✗
System.out.println(new Key("A").equals(new Key("A"))); // true  — yet unfi
System.out.println(map.size());                        // 1     — it IS in

Set<Key> set = new HashSet<>();
set.add(new Key("A"));
set.add(new Key("A"));
System.out.println(set.size());                        // 2     ✗ duplicat
```

The correct implementation, in the form worth memorising:

JAVA Copy

```
public final class Point {
    private final int x;
    private final int y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
```

```
        Point p = (Point) o;
        return x == p.x && y == p.y;                    // compare the same f
    }                                                   // hashCode uses (see

    @Override
    public int hashCode() {
        return Objects.hash(x, y);                      // same fields, same
    }
}
```

Note that `instanceof` already returns `false` for `null`, so it handles the non-nullity requirement for free — one reason it is preferred over `getClass() ≠ o.getClass()` in most code. Java 16's pattern matching collapses the test and cast into one line:

JAVA Copy

```java
@Override
public boolean equals(Object o) {
    return this == o || (o instanceof Point p && x == p.x && y == p.y);
}

// For most classes, stop hand-writing this entirely — a record generates
// a correct, consistent pair for you:
public record Point(int x, int y) { }
```

The mutable-key trap, which is the second classic failure:

JAVA Copy

```java
class MutableKey {
    String id;                                  // NOT final — the problem
    MutableKey(String id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return o instanceof MutableKey k && Objects.equals(id, k.id);
    }
    @Override public int hashCode() { return Objects.hash(id); }
}
```

```
map.put(key, "value");

key.id = "B";                    // mutate a field equals/hashCode depends

System.out.println(map.get(key));    // null — new hash, wrong bucket
System.out.println(map.containsKey(key)); // false
// The entry is still in the table, in the bucket for the OLD hash, and is
// unreachable by any key. Hash keys must be effectively immutable.
```

Under the hood

Why `Objects.hash` is not always the right answer. It is varargs, so it allocates an `Object[]` on every call and boxes primitives. In a hot path — a key used in millions of map lookups — that is measurable, and the hand-rolled form avoids it:

JAVA Copy

```java
@Override
public int hashCode() {
    int result = 17;
    result = 31 * result + x;
    result = 31 * result + y;
    return result;
}
// 31 is the traditional multiplier: odd (so no information is lost to
// shifting), prime, and 31 * i compiles to (i << 5) - i. Use Objects.hash
// for clarity by default, and this form only where profiling says it matt
```

The symmetry trap with inheritance is the hardest part of this topic and a favourite senior question. If `Point` uses `instanceof` and `ColorPoint extends Point` adds a colour to its `equals`, then `point.equals(colorPoint)` is true while `colorPoint.equals(point)` is false — symmetry is broken, and collections behave unpredictably. There is no fully satisfying fix: `getClass()` comparison restores symmetry but breaks Liskov substitution (a subclass instance is never equal to a base instance, which also breaks Hibernate proxies). The honest answer is Joshua Bloch's —

Never use a hash code as an identifier. It is 32 bits, not unique, and not stable across JVM runs for the default implementation. It is a bucket hint, nothing more.

Common mistakes

- **Overriding `equals` without `hashCode`.** The headline bug: objects vanish inside hash collections.
- **Writing `equals(MyType o)`.** That overloads rather than overrides. `@Override` catches it instantly.
- **Using different fields in each method.** They must be derived from the same state, or the contract breaks.
- **Mutating a field used by `hashCode` after inserting into a map.** The entry becomes unreachable. Keys must be effectively immutable.
- **Returning a constant from `hashCode`.** Technically legal, and it degrades every hash map to a linked list.
- **Using `==` to compare fields that are objects.** Use `Objects.equals(a, b)`, which is also null-safe.
- **Including derived or transient fields.** A cached total or a timestamp makes equality unstable.
- **Treating a hash code as an id.** Collisions are normal and expected.

Interview lens

> **Q: "What is the contract between `equals` and `hashCode`?"**
> A: If two objects are equal by `equals`, they must return the same `hashCode`. The reverse is not required — unequal objects may collide. Both must be computed from the same fields, and `equals` must additionally be reflexive, symmetric, transitive, consistent, and false for null.

A: Hash-based collections break. Two objects that are equal get different identity hash codes, so they map to different buckets. `map.get(equalKey)` returns null even though the entry is there, and a `HashSet` will happily store duplicates. It is a silent bug — nothing throws.

Q: "And the reverse — `hashCode` without `equals`?"

A: Less catastrophic but still broken. Equal-in-meaning objects land in the same bucket, then `equals` falls back to identity comparison and says they are different — so lookups still fail and sets still hold duplicates. You need both.

Q: "Why 31 in hash code implementations?"

A: It is odd and prime, so multiplying does not throw away information the way an even factor's shifting would, and it spreads values reasonably. It also optimises well — `31 * i` becomes `(i << 5) - i`. It is convention with sound reasons, not a magic number.

Q: "Can two unequal objects have the same hash code?"

A: Yes, and it is expected — a hash code is 32 bits and the space of objects is unbounded, so collisions are inevitable. That is why the contract only runs one way, and why a hash map always calls `equals` after finding the bucket. Frequent collisions are only a performance problem, not a correctness one.

Q: "What if a key is mutated after being put in a map?"

A: If the mutation touches a field `hashCode` depends on, the entry becomes unreachable — it sits in the bucket for the old hash while lookups compute the new one. It will not even be found by the very same key object. This is why map keys should be immutable, and why `String` is such a good key.

A: `instanceof` for most code — it is null-safe and keeps subclass instances comparable, which matters for things like Hibernate proxies. Its weakness is that a subclass adding fields to `equals` breaks symmetry. `getClass()` restores symmetry but violates Liskov substitution. The real fix is to avoid inheritance for value types — make them `final`, or use a record.

Q: "How do you implement these today?"
A: Usually I do not — a record generates a correct, consistent `equals`, `hashCode` and `toString` from its components. Where I need a class, I use pattern-matching `instanceof` plus `Objects.hash`, and I make the class final and its fields final so the key can never drift.

Cheat-card

- **Equal ⟹ same hashCode.** Same hashCode does NOT imply equal.
- Override **both, from the same fields**, or hash collections break silently.
- `equals` must be reflexive, symmetric, transitive, consistent, false-for-null.
- Lookup = **hashCode picks the bucket, equals confirms the key**.
- Keys must be **effectively immutable** — mutating one strands the entry forever.
- `@Override` always: `equals(MyType)` is an overload, not an override.
- `Objects.hash( ... )` for clarity; hand-rolled `31 *` form only in hot paths.
- `instanceof` over `getClass()`; symmetry breaks with subclass fields either way → make value classes `final`.
- Best answer in modern Java: **use a record**.

QUICK CHECK — no marks, just for you.

1. You override equals() but not hashCode(). You put a key in a HashMap and look it up with an equal key. What happens?

It returns null — the two keys produce different hash codes and map to different buckets, so equals is never even consulted

- It returns the value, because equals() is what a HashMap uses
- It throws IllegalStateException
- It works, but the map is slower

Check answer

2. Can two objects that are NOT equal share a hash code?

- Only if they are instances of different classes
- Yes, and it is expected — a hash code is 32 bits over an unbounded object space, so collisions are inevitable
- Only when hashCode() is badly implemented
- No — that would violate the contract

Check answer

3. Why is 'implement equals(MyType other)' a bug?

- It breaks serialisation
- It is not — that is the type-safe way to write it
- It overloads Object.equals(Object) rather than overriding it, so every library that calls equals(Object) — including HashMap — ignores it
- It causes infinite recursion

Previous
Encapsulation and Access Modifiers

Next
static, final and Initialisation Order

LEARN MORE

ARTICLES & DOCS

- Object.equals and Object.hashCode — API docs
  docs.oracle.com
- Objects utility class — API docs
  docs.oracle.com

VIDEOS



- Equals and Hashcode Contract in Java [Important Java Interview Question] | Code Decode
  Code Decode



- Java HashCode Equals: Interview Guide
  KnoDAX

Hand-picked references for this lesson.